1. AI Foundations and Biomedical Context

Course map and central idea

- **Artificial Intelligence (AI)** aims to perform tasks associated with intelligence; **machine learning (ML)** learns patterns from data; **neural networks (NNs)** are trainable layered function approximators; **deep learning** uses many layers; **generative AI** produces new content from learned distributions.
- The modern workflow is **training** on examples followed by **inference** on new inputs. Scaling usually means more data, parameters and compute.
- **Evolution of neural networks:** McCulloch & Pitts neuron (1943); perceptron (1957); Multilayer perceptron/Backpropagation(1986); Convolutional neural networks(1989); Alexnet(2012); Transformers(2017).
- Course progression: foundations → Python and learning paradigms → supervised learning → NNs → deep learning/CNNs → language models and Transformers.

Two traditions of AI

Approach	Core belief	Typical methods
Knowledge engineering	Intelligence is encoded explicitly as facts, rules and logic	Expert systems, symbolic reasoning, hand-crafted features
Statistical learning	Intelligence emerges from statistical patterns in data at scale	Regression, probabilistic models, NNs, hidden Markov models

- Rule-based systems are interpretable but expensive to author and maintain. Statistical learning scales better with data and computing, but depends on data quality, objectives and validation.
- Other computational-intelligence methods include genetic algorithms, particle swarm optimization and simulated annealing.

AI development milestones

- Artificial neurons abstract biological neurons as weighted input summation plus an activation. Stacked neurons can approximate complex functions.
- Backpropagation made multilayer-network training practical. **AlexNet (2012)** demonstrated the impact of large labeled datasets and GPUs; **Transformers (2017)** enabled efficient long-range sequence modeling; **ChatGPT (2022)** accelerated public adoption of generative AI.
- Progress is not only an algorithmic story: internet-scale data, specialized hardware and larger models are major drivers.

Biomedical applications and limits

- AI can support imaging, diagnosis, prognosis, drug discovery, genomics, protein/DNA modeling and personalized therapy. It should be treated as a tool within a validated clinical workflow, not as a substitute for domain expertise.
- **Personalized mRNA cancer vaccine**: tumor and blood samples are sequenced to identify patient-specific mutations (neoantigens); algorithms help rank targets and optimize mRNA design. The therapy stimulates T cells against residual tumor cells and was combined with an immune-checkpoint inhibitor.
- AI did not independently invent the vaccine or cure cancer. Biology, clinical trials, manufacturing and safety validation remain essential.
- A recurrence-free-survival result after surgery in patients who already have melanoma does not show that the vaccine prevents cancer in healthy people. Melanoma also has a relatively high mutation burden and responds well to immunotherapy, so success cannot be assumed for colorectal, breast or lung cancer without further evidence.
- mRNA design targets codon usage, UTRs, poly(A)-tail length and capping to improve stability, translation and protein expression; lipid nanoparticles deliver the mRNA.
- **Eroom's law**: the number of new drugs approved per billion US dollars spent on R&D has halved roughly every nine years since 1950 despite improving technology. This is the reverse of **Moore's law**, in which computing capability has grown rapidly; drug development has historically become slower and more expensive. AI may improve parts of the pipeline, but it is not a magic wand.

Practical and ethical principles

- Use AI as a tutor/collaborator only when allowed; never use it in exams or submit work that cannot be reproduced and explained.
- Biomedical models require representative data, correct labels, leakage prevention, external validation, calibration, robustness checks, interpretability where appropriate, privacy protection and human oversight.

2. Learning Paradigms and Python

Structure of AI training

- **Training** usually involves two ways, One encodes rules by hand, as in expert systems. The other lets models learn patterns from data. The latter approach underlies most modern AI models.
- One trend in the evolution of AI is scaling up: more training data; more model parameters; more computing resources for training.

Supervised versus unsupervised learning

- **Supervised learning**: each input has a target label. Learn $f : x \mapsto y$, then predict labels for unseen inputs.
- **Regression**: y is continuous (e.g., predict blood glucose from BMI). **Classification**: y is categorical (e.g., Type 1 versus Type 2 diabetes).
- **Unsupervised learning**: inputs are unlabeled; the model discovers structure such as clusters, similarity or low-dimensional representations.

Question	Supervised	Unsupervised
Training signal	Known correct answers	No target answers
Typical output	Prediction/class label	Structure, clusters or representation
Biomedical example	Glucose prediction, tumor classification	Patient subtypes or cell-state clusters

Python and Jupyter

- Python is interpreted rather than compiled, but dominates AI research because of its open ecosystem and mature scientific packages.
- Package organization is commonly `package > module > class/function`; capabilities are added with `import`.
- A Jupyter notebook (`.ipynb`) combines executable code, Markdown notes and saved outputs. Cells can be run incrementally, but execution order matters.
- Conda/Mamba manage isolated environments: each project can have its own Python interpreter and packages, preventing dependency conflicts.

Lab habit: inspect data, document assumptions, set random seeds where appropriate, separate training and test data, and keep the computational environment reproducible.

3. Supervised Learning

Linear regression

- Model a continuous response with a line: $\hat{y} = f(x) = wx + b$. Here w is the slope and b the intercept.
- For sample i, error is $E^{(i)} = \hat{y}^{(i)} - y^{(i)}$. A common objective is mean squared error (MSE):

$$J(w,b) = \frac{1}{2m}\sum_{i=0}^{m-1}\left(f(x^{(i)}) - y^{(i)}\right)^2.$$

- Training chooses w, b to minimize J. Ordinary least squares has a closed-form solution for basic linear regression; gradient descent provides an iterative alternative and generalizes to larger models.

Gradient descent

- The gradient gives the direction and rate of increase of cost. Move in the opposite direction:

$$w_{t+1} = w_t - \alpha\frac{\partial J}{\partial w}, \quad b_{t+1} = b_t - \alpha\frac{\partial J}{\partial b}.$$

- α is the learning rate. Too small gives slow training; too large can overshoot, diverge or oscillate. With a suitable α, the cost decreases toward a minimum.
- Generic loop: initialize parameters → compute predictions, cost and gradients → update parameters → repeat until a stopping criterion.
- On a convex linear-regression cost surface, training from different initial w, b values converges to the same global minimum when the learning rate is suitable: the cost decreases over iterations and the parameter path moves downhill on the cost surface. A large learning rate can oscillate or diverge; a small one converges slowly.

Logistic regression and classification

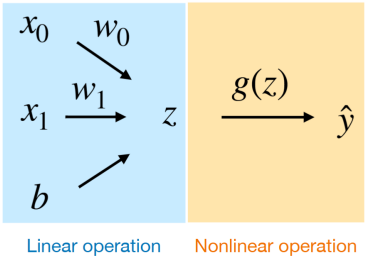
- A linear score is converted to a probability with the sigmoid:

$$z = w^\top x + b, \qquad \sigma(z) = \frac{1}{1+e^{-z}}.$$

- Threshold the predicted probability to obtain a class. Binary classification commonly uses cross-entropy (log loss), which strongly penalizes confident wrong predictions:

$$L = -\left[y\log\hat{y} + (1-y)\log(1-\hat{y})\right].$$

- Logistic regression is a single artificial neuron: weighted sum plus bias, followed by a nonlinear activation. Decision boundaries are linear in the input features.



4. Neural Networks

Artificial neuron and perceptron

- A biological neuron receives signals through dendrites, integrates them in the soma and sends an action potential through the axon. An artificial neuron simplifies this to

$$z = \sum_j w_j x_j + b, \qquad \hat{y} = g(z).$$

- A perceptron learns weights from prediction errors and can implement AND and OR logic. It can separate only **linearly separable** classes with one hyperplane.
- XOR is not linearly separable: no single straight decision boundary correctly separates its four input combinations. This motivates hidden layers.

Multilayer perceptron (MLP)

- Neurons form layers: input → hidden layer(s) → output. Hidden layers transform features so that nonlinear decision boundaries become learnable.
- **Forward propagation**: pass inputs through each layer to obtain $\hat{y}$. Compute loss/cost from predictions.
- **Backpropagation**: apply the chain rule from output to input to obtain gradients for all weights and biases, then update them with gradient descent. Frameworks such as Keras and PyTorch calculate gradients automatically.

forward pass → loss → backward pass → parameter update.

Breast-cancer classification lab

- Inputs are cell-nucleus measurements extracted from biopsy images; targets are benign or malignant.
- Compare a single-layer perceptron (equivalent to logistic regression for binary classification) with a one-hidden-layer network.
- Split data into training and test sets. Standardize features so differing numerical scales do not let one feature dominate optimization.
- A hidden layer can improve accuracy by learning nonlinear feature combinations, but more capacity also increases overfitting risk. Evaluate on unseen data.

5. Deep Learning and Convolution

Activations and deep representations

- Deep networks stack layers to learn hierarchical representations: early layers detect simple patterns; later layers combine them into abstract features.
- Sigmoid saturates at large positive/negative inputs, producing near-zero gradients. ReLU is

$$\mathrm{ReLU}(z) = \max(0, z)$$

- ReLU responds to positive inputs with a nonzero linear gradient and is generally easier to optimize in deep networks.

Convolutional neural networks (CNNs)

- Dense layers connect every input to every hidden unit. CNNs replace this with **sparse local connections**, reducing parameters and exploiting local structure.
- A kernel/filter slides across an input and computes a local dot product, producing a feature map. Kernels are **shared weights**: the same learned filter detects a pattern at different positions.
- Stacked layers build representations from edges/lines to textures/shapes to cells and tissue morphology. This matches the visual pathway, where neurons respond to local features and progressively more complex combinations.

$$\text{feature map position} = \sum_{\text{local patch}} (\text{input value})(\text{kernel weight}) + \text{bias}.$$

- CNNs are parameter-efficient and well suited to images, microscopy, spatial signals and short local motifs in sequences.

Deep learning in genomics

- DNA is sequence data; one-hot encoding represents A, C, G, T as basis vectors. A 1D CNN can scan a sequence for local motifs such as regulatory patterns.
- The main lesson is to match architecture to data structure: convolution captures local motifs, while sequence models are needed when distant positions interact.

AlexNet and the modern deep-learning turning point

- AlexNet's 2012 ImageNet result marked a practical turning point for deep learning. Its success depended on **large-scale labeled data** (ImageNet), **GPUs** that made training feasible, and a **CNN architecture** that exploited image locality and shared weights.
- Deep networks learn increasingly abstract hierarchical representations, while CNNs achieve this with far fewer parame-

ters than fully connected networks. Their limitation is that small kernels capture local patterns and require depth to connect distant positions.

6. Language Models and Transformers

Why sequences are different

- Order matters in text, speech, DNA, proteins and time series; sequences may have variable length and long-range dependencies.
- MLPs do not naturally represent order. CNNs capture local context, but a distant dependency may require many layers and a long receptive-field path.
- Example: in "Dr. Alvarez ... finally ___ awarded", the verb must agree with the distant singular subject "Alvarez", not nearby plural distractors.

Transformer input representation

1. **Tokenization**: split input into words, subwords, amino acids, nucleotides or k-mers. DNA may use single bases ($k = 1$) or fixed-length k-mers.
2. **Token embedding**: map token IDs to learned dense vectors. Unlike one-hot vectors, embeddings can encode useful relationships.
3. **Positional encoding**: add position information because attention alone does not preserve order.
4. The resulting order-aware vectors enter the Transformer layers.

Self-attention

- For each token, Query Q, Key K and Value V vectors are computed. A query compares with all keys to obtain relevance weights, then forms a weighted sum of values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

- Every token can directly attend to every other token, so a long-range relationship can be modeled in one attention hop. Transformer layers pair attention with a feed-forward network; stacking layers builds richer representations.
- Unlike recurrent models, attention can process positions in parallel during training. The trade-off is high memory/computational cost for very long sequences.

Biomedical sequence models

- Transformers power large language models and also apply to proteins, DNA, audio, video and time series.
- A DNA language model can be trained to predict a masked or next token and then fine-tuned for tasks such as promoter prediction. A protein model can tokenize amino acids plus special start/end tokens.
- Architecture selection: CNNs are efficient for local motifs; self-attention is effective for interactions across long genomic or biological sequences.

7. Unsupervised Learning and Generation

Principal component analysis (PCA)

- PCA finds orthogonal linear directions (principal components) that maximize projected variance, without using labels. The first component captures the greatest variance; later components capture the greatest remaining variance subject to orthogonality.
- Projecting samples onto the first few components gives a low-dimensional representation for visualization and compression. The retained components explain a fraction of the total variance; discarded components cause reconstruction error.
- In biomedical data, many correlated measurements (e.g., thousands of RNA-seq gene features) can often be summarized by a few PC scores. PCA is linear, so it may miss curved or branching nonlinear structure such as cell-differentiation trajectories.

Autoencoders and latent spaces

- An autoencoder learns input $\rightarrow$ encoder $\rightarrow$ latent bottleneck $\rightarrow$ decoder $\rightarrow$ reconstruction. The input is also the training target, so labels are not required.
- The latent space is a learned, generally nonlinear low-dimensional representation; the decoder reconstructs the original profile. Compared with PCA, an autoencoder can represent nonlinear patterns, but a standard autoencoder's latent space is not necessarily organized for freely generating realistic new samples.

Autoregressive generation

- A Transformer can generate a sequence by predicting the next token from the prefix, appending the prediction, and repeating. Causal (masked) self-attention prevents a position from using future tokens, making next-token prediction self-supervised.

8. Exam and Project Checklist

- Distinguish AI, ML, NN, deep learning and generative AI; distinguish supervised regression, supervised classification and unsupervised learning.
- Be able to write and interpret $y = wx + b$, MSE, sigmoid, cross-entropy, gradient-descent updates, ReLU and attention.
- Explain why a perceptron handles AND/OR but not XOR; how hidden layers solve nonlinear separation; and how forward propagation/backpropagation train an MLP.
- Explain CNN locality and weight sharing, receptive fields and the biological inspiration of hierarchical visual features.
- Explain tokenization, embeddings, positional encoding, $Q/K/V$ attention and why Transformers handle long-range dependencies.
- For a CNN synthesis answer, connect AlexNet's ImageNet data, GPU training and CNN architecture to deep hierarchical features, parameter efficiency, the receptive-field limitation, and the direct long-range connections provided by self-attention.
- Be able to identify the first principal component as the direction of maximum variance, distinguish PCA from an autoencoder, and describe autoregressive next-token generation.
- For any biomedical AI claim, ask: What is the biological target? What data and labels are used? What is the split and baseline? How is leakage prevented? Which metric and external validation support the claim? What are failure modes and clinical consequences?